

FECAP

FUND.



1902

FECAP

Estrutura de Dados

- AULA 02



Estrutura de Dados

Agenda

Análise de Complexidade

Revisão de Vetores (Array)

Algoritmos e Estrutura de Dados

COMPLEXIDADE DE ALGORITMOS

Uma característica importante de qualquer algoritmo é o seu **tempo de execução**.

Considere um problema qualquer e um algoritmo (como uma sequência finita de passos executáveis que visam obter uma solução para um determinado tipo de problema), podemos determinar o seu tempo de execução duas formas:

Empírica (Experimentalmente): Executa-se o algoritmo em um computador e medimos o tempo de execução em milissegundo ou segundos.

Analítica: Identificar a quantidade de passos executáveis necessários para resolver o problema.

Estrutura de Dados

COMPLEXIDADE DE ALGORITMOS

Empírica (Experimentalmente): O tempo de execução depende do computador, um mesmo algoritmo pode levar mais ou menos tempo dependendo da velocidade do computador.

Analítica: Independe do computador, se um algoritmo A qualquer leva n passos para resolver um problema e um algoritmo B qualquer leva m passos para resolver o mesmo problema, n é menor que m , dizemos então que o algoritmo A é mais eficiente que o algoritmo B , de outra forma, dizemos que A tem um GRAU DE COMPLEXIDADE MENOR que B .

Estrutura de Dados

COMPLEXIDADE DE ALGORITMOS

Exemplo 1: Considere o algoritmo .que resolva o problema de achar o maior número entre 3 números dados.

```
início
  Leia (A)
  Leia (B)
  Leia (C)
  Maior=A
  Se B>Maior então Maior=B
  Se C>Maior então Maior=C
  Imprimir (Maior)
fim
```

Podemos dizer que esse algoritmo executa 7 passos para a solução do problema, ou seja, esse algoritmo é de Tempo 7.

Estrutura de Dados

COMPLEXIDADE DE ALGORITMOS

Exemplo 2: Considere agora o algoritmo que some os N primeiros números inteiros

```
início  
  Leia(n)  
  soma=0  
  Para i de 1 até n  
    soma=soma+i  
  Imprimir(soma)  
fim
```

Note que o passo *soma*=*soma*+*i* é executado *n* vezes, isto é, a quantidade necessárias de passos para solução do problema varia de acordo com *n*. Dessa forma podemos dizer que esse algoritmo é de Tempo $n + 3$. Note que para *n* grande (10.000 por exemplo) a CONSTANTE 3 torna-se IRRELEVANTE, dessa forma dizemos que esse algoritmo tem tempo de **ORDEM *n***, ou simplesmente **$O(n)$** (dizemos O de *n*). Da mesma forma, no primeiro exemplo dizemos que o tempo é **CONSTANTE**, ou seja, **$O(1)$** .

Estrutura de Dados

COMPLEXIDADE DE ALGORITMOS

Notação **O** (Chamada de *big O*)

Formalmente a notação O é definida da forma:

Seja $f(n)$ e $h(n)$ funções reais não negativas da variável inteira $n \geq 0$, dizemos que $f(n) = O(h(n))$ quando existir uma constante $c > 0$ e um valor inteiro n_0 tal que para qualquer $n > n_0$ temos que:

$$f(n) \leq c \cdot h(n)$$

Exemplo: Seja a função $f(n) = 10n^3 + 15$, dizemos então que $f(n) = O(n^3)$

Estrutura de Dados

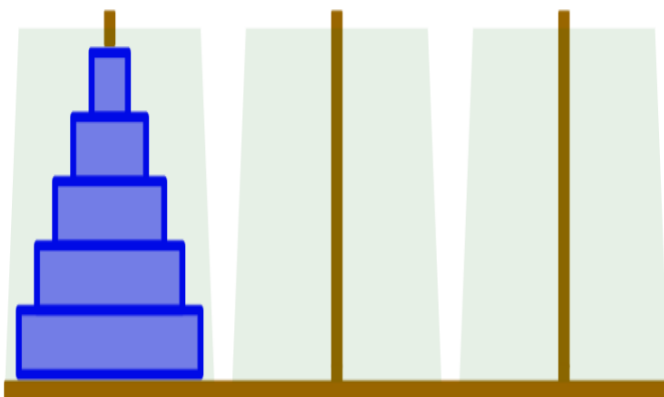
COMPLEXIDADE DE ALGORITMOS

Problema da Torre de Hanoi

<https://www.novelgames.com/pt/tower/>

<http://clubes.obmep.org.br/blog/torre-de-hanoi/>

https://panda.ime.usp.br/panda/static/pythonds_pt/04-Recursao/09-hanoi.html



A quantidade mínima de movimentos é dada por $2^n - 1$ movimentos, ou seja $O(2^n)$, dessa forma temos:

$n=3 \Rightarrow 7$ Movimentos

$n=4 \Rightarrow 15$ Movimentos

$n=5 \Rightarrow 31$ Movimentos

$n=64 \Rightarrow ???$ Movimentos

Dizemos que o crescimento é EXPONENCIAL

Estrutura de Dados

Vetor (Array) em C#

Um *ARRAY* de uma dimensão, também chamado de *VETOR*, tem o comportamento de uma *LISTA INDEXADA*, isto é, dado um índice *i* qualquer, sendo *i* uma posição válida na lista, conseguimos acessar o ***i-ésimo*** elemento diretamente, sem a necessidade de percorrer a lista do início até a posição *i*.

Características de um VETOR

- ✓ Array unidimensional
- ✓ Seus elementos são armazenados de maneira sequencial (Obs. Em POO pode ter elemento Nulo)
- ✓ Cada elemento pode ser acessado diretamente utilizando um índice (índex)
- ✓ No C# todos os elementos em um vetor são do mesmo tipo

Estrutura de Dados

Vetor (Array) em C#

Implementando um Vetor de Inteiros em C# e acessando o i-ésimo elemento

```
const Int32 N = 10;
Int32[] lista = new Int32[N];
Random rnd = new Random();
for (int i = 0; i < lista.Length; i++)
{
    lista[i] = rnd.Next(0, 100);
}

for (int i = 0; i < lista.Length; i++)
{
    Console.WriteLine(lista[i]);
}
```

Estrutura de Dados

Vetor (Array) em C#

Implementando um Vetor de Inteiros em C# e acessando o i-ésimo elemento

```
const Int32 N = 10; //Declara uma constante inteira N igual 10
Int32[] lista = new Int32[10]; //Declara e instancia um vetor de Inteiros de N posições
Random rnd = new Random();
for (int i = 0; i < lista.Length; i++)
{
    lista[i] = rnd.Next(0, 100); //Coloca na i-ésima posição um inteiro aleatório
}

for (int i = 0; i < lista.Length; i++)
{
    Console.WriteLine(lista[i]); //Imprime o i-ésimo elemento
}
}
```

Quantos passos são necessários para execução desse programa(algoritmo)?

Qual a Ordem de Grandeza do tempo de execução (complexidade) deste programa em notação **O** (Big O)?

Estrutura de Dados

Funções ou Métodos em C#

Um método (ou função) é um bloco de código que contém uma série de instruções. Um programa faz com que as instruções sejam executadas chamando o método e especificando os argumentos de método necessários. No C#, todas as instruções executadas são realizadas no contexto de um método.

```
public static void imprimirVetor(Int32[] vetor){  
    for (int i = 0; i < vetor.Length; i++)  
    {  
        Console.WriteLine(vetor[i]); //Imprime o i-ésimo elemento  
    }  
}
```

Estrutura de Dados

Funções ou Métodos em C#

Refaça agora o primeiro código utilizando a função criada

```
Int32[] vetor = new Int32[10];

imprimirVetor(vetor);
Random rnd = new Random();
for (int i = 0; i < vetor.Length; i++)
{
    vetor[i] = rnd.Next(0, 100);
}

imprimirVetor(vetor);
}
```

Algoritmos e Estrutura de Dados

Funções ou Métodos em C#

Exemplo de Simulação para a função imprimirVetor(vetor) para uma vetor = [1,3,5,7,9,2,4,6,8,10]

Lista	0	1	2	3	4	5	6	7	8	9
	1	3	5	7	9	2	4	6	8	10
	i	lista.length	lista[i]							
	0	10	1							
	1		3							
	2		5							
	3		7							
	4		9							
	5		2							
	6		4							
	7		6							
	8		8							
	9		10							
	10									



Algoritmos e Estrutura de Dados

Funções ou Métodos em C#

Faça uma função gerarVetor(N) que gere um vetor de tamanho N de números inteiros de aleatórios de forma que nossa programa fique:

```
Int32[] vetor = new Int32[10];
```

```
imprimirVetor(vetor);  
lista=gerarVetor(10);  
imprimirVetor(vetor);
```


FECAP



(11) 3272-2222 / www.fecap.br

Av. Liberdade, 532 / 01502-001 / São Paulo - SP

Fundação Escola de Comércio Álvares Penteado - **FECAP**